

TDE — Detecção e recuperação de impasses

Programação Concorrente: Filósofos, Semáforos e Deadlock

Linguagens permitidas: Java ou Python

AVISO IMPORTANTE — LEIA ANTES DE COMEÇAR

É terminantemente proibido o uso de Inteligência Artificial generativa (ChatGPT, Claude, Gemini, Copilot, Cursor, DeepSeek, Llama, ferramentas de geração de vídeo/voz/legendas automáticas etc.) para a confecção dos códigos, das soluções escritas, dos roteiros, das narrações ou dos vídeos. Qualquer evidência de uso de IA generativa em qualquer etapa do trabalho acarretará o **zeramento integral da atividade** para todos os integrantes do grupo, sem possibilidade de reentrega. O professor poderá realizar arguições orais individuais para confirmar autoria, e a incapacidade de explicar qualquer parte do código ou da solução também caracteriza descumprimento desta regra.

Esta atividade equivale a 40% da nota do RA.

Grupos de no máximo 4 pessoas.

Entrega 17/06

Visão geral

Este TDE explora três problemas centrais da programação concorrente — o Jantar dos Filósofos, condições de corrida controladas por semáforos e o tratamento de deadlock — exercitando, na prática, as quatro condições de Coffman e as estratégias clássicas de prevenção e correção.

O trabalho é composto por três partes complementares. Em cada uma o grupo deve (a) compreender o problema, (b) implementar a solução em Java ou Python e (c) explicar verbalmente, em vídeo, o que foi feito e por quê.

Parte 1 — Jantar dos Filósofos

O Jantar dos Filósofos modela cinco filósofos sentados em uma mesa circular, alternando entre pensar e comer, onde para comer cada filósofo precisa dos dois garfos à sua esquerda e direita, compartilhados com os vizinhos, expondo problemas clássicos de exclusão mútua, impasse e inanição em sistemas concorrentes.

Um impasse ocorre se todos pegarem simultaneamente o garfo da esquerda e aguardarem o da direita, pois ninguém progride e todos esperam indefinidamente, caracterizando deadlock no protocolo ingênuo de “pegar primeiro um garfo, depois o outro”.

Soluções gerais exigem negar pelo menos uma das quatro condições necessárias para deadlock — exclusão mútua, manter-e-esperar, não preempção e espera circular — sendo comum eliminar a espera circular por meio de hierarquia de recursos, usar um árbitro (garçom) ou limitar a $N-1$ filósofos ativos.

Objetivos

Implementar (em Java ou Python) a simulação do problema com $N = 5$ (ou seja, 5 filósofos), registrando em log/console os estados “pensando”, “com fome” e “comendo” de cada filósofo, evidenciando empiricamente como o protocolo ingênuo pode levar a impasse.

Projetar e implementar um protocolo que evite impasse (por exemplo, hierarquia de recursos, árbitro/garçom, ou limite de $N-1$ filósofos ativos), justificando logicamente qual das condições de Coffman é negada pela estratégia escolhida.
Garantir progresso e justiça (ausência de inanição), discutindo brevemente como a solução evita que um mesmo filósofo coma sempre antes dos demais.

Entregáveis da Parte 1

Código-fonte da simulação (Java OU Python) contendo: (i) versão ingênua que pode entrar em deadlock e (ii) versão corrigida sem deadlock.
Relatório curto (no README do repositório) explicando a dinâmica do problema, por que o impasse surge e qual condição de Coffman será negada na solução proposta.
Pseudocódigo ou fluxograma do protocolo, demonstrando a ordem de aquisição e liberação de recursos e como justiça/progresso são preservados.

Pseudocódigo de referência (hierarquia de recursos)

Atribua um índice único a cada garfo e force todos os filósofos a requisitarem primeiro o garfo de menor índice, depois o de maior índice, removendo a espera circular ao impor uma ordem parcial fixa sobre os recursos.

Dados:

```
N = 5 filósofos
Garfos 0..N-1 (garfo i fica entre filósofos i e (i+1) mod N)
```

Para cada filósofo p:

```
left  = min(garfo_esquerda(p), garfo_direita(p))
right = max(garfo_esquerda(p), garfo_direita(p))
```

Loop:

```
pensar()
estado[p] <- "com fome"
adquirir(left)    // bloqueia até o garfo estar livre
adquirir(right)   // bloqueia até o garfo estar livre
estado[p] <- "comendo"
comer()
liberar(right)
liberar(left)
estado[p] <- "pensando"
```

O protocolo evita impasse porque elimina a condição de espera circular ao impor uma ordem global, garantindo que não possa existir um ciclo de espera entre filósofos.

Parte 2 — Threads e semáforos

Um semáforo controla o acesso a um recurso compartilhado por meio de contagem de “permissões”: adquirir bloqueia quando não há permissões e liberar devolve permissões. Quando inicializado com uma única permissão (binário), funciona como mecanismo de exclusão mútua.

Tanto em Java quanto em Python existem implementações nativas:

Java: `java.util.concurrent.Semaphore`, com modos justo e não justo, métodos `acquire/release` e garantias `happens-before` entre `release` de uma thread e `acquire` de outra.
Python: `threading.Semaphore` e `threading.BoundedSemaphore`, com métodos `acquire/release`. Atenção ao GIL: prefira realizar trabalho não trivial dentro da seção crítica

ou execute o experimento com operações suficientemente custosas para que a corrida seja observável.

Desafio: contador concorrente

Objetivo: demonstrar uma condição de corrida incrementando um contador compartilhado a partir de múltiplas threads sem sincronização e, em seguida, corrigi-la com um semáforo binário, comparando o valor final e o tempo de execução das duas versões.

Passos sugeridos: criar um contador compartilhado; lançar T threads, cada uma executando M incrementos; medir o valor final e o tempo gasto nas duas versões (com e sem semáforo); discutir diferenças, justiça (fairness) e impacto no throughput.

Parâmetros mínimos sugeridos para que a corrida fique evidente: $T \geq 8$ threads e $M \geq 200.000$ incrementos por thread (ajuste conforme a linguagem para garantir reprodutibilidade).

Entregáveis da Parte 2

Código com duas versões executáveis: (i) sem sincronização, evidenciando a perda de incrementos, e (ii) com semáforo binário, produzindo o valor correto $T \times M$.

Tabela de resultados no README com pelo menos 3 execuções de cada versão: valor esperado, valor obtido, tempo de execução.

Discussão escrita sobre: por que a versão sem sincronização perde incrementos, por que a versão com semáforo é correta, o trade-off de throughput e o conceito de visibilidade/ordenação (happens-before em Java; barreiras implícitas em Python).

Pseudocódigo de referência

Globais:

```
count = 0
sem = Semaforo(permissoes = 1, justo = true) // binário com FIFO
```

Funcao tarefa():

```
para i de 1 ate M:
    // VERSAO 1 – sem sincronizacao (condicao de corrida):
    count = count + 1

    // VERSAO 2 – com semaforo (correta):
    sem.adquirir()
    try:
        count = count + 1
    finally:
        sem.liberar()
```

Programa principal:

```
iniciar T threads executando tarefa()
esperar todas terminarem
imprimir esperado = T*M, obtido = count, tempo = ...
```

Parte 3 — Deadlock

Deadlock é a situação em que um conjunto de processos ou threads não progride porque cada um aguarda uma ação do outro, tipicamente a liberação de um lock. Para ocorrer, as quatro condições de Coffman devem se manifestar simultaneamente: exclusão mútua, manter-e-esperar, não preempção e espera circular.

Estratégias clássicas de tratamento incluem: ignorar (ostrich), detectar e recuperar, prevenir (quebrar uma das condições — por exemplo, impedindo manter-e-esperar ou impondo uma hierarquia de recursos) e evitar (análise antecipada de alocações seguras, como o algoritmo do Banqueiro).

Atividade com deadlock

Cenário a reproduzir: duas threads e dois locks (A e B). A Thread 1 adquire A e depois B; a Thread 2 adquire B e depois A. Quando cada uma segura um lock e aguarda o outro, surge espera circular e o sistema trava.

Objetivo: (i) reproduzir o deadlock de forma observável, com logs que mostrem a ordem de aquisição e o travamento; (ii) explicar quais condições de Coffman se manifestaram; (iii) implementar uma correção, preferencialmente impondo ordem global de aquisição de recursos (hierarquia).

Entregáveis da Parte 3

Código que reproduz o deadlock (pode ser necessário usar um pequeno `Thread.sleep / time.sleep` entre as aquisições para tornar a corrida determinística).

Logs ou prints anexados ao README evidenciando o travamento (programa que não termina e/ou diagnóstico do tipo `jstack/threading enumerate` apontando as threads bloqueadas).

Código corrigido aplicando ordem global de aquisição (sempre `LOCK_A` antes de `LOCK_B`), com explicação de qual condição de Coffman foi quebrada.

Pseudocódigo de referência (versão que trava)

`LOCK_A, LOCK_B = locks distintos`

Thread 1:

```
adquirir(LOCK_A)
dormir(50ms)           // aumenta a chance do deadlock
adquirir(LOCK_B)
imprimir "T1 concluiu"
liberar(LOCK_B)
liberar(LOCK_A)
```

Thread 2:

```
adquirir(LOCK_B)
dormir(50ms)
adquirir(LOCK_A)
imprimir "T2 concluiu"
liberar(LOCK_A)
liberar(LOCK_B)
```

Pseudocódigo de referência (versão corrigida — hierarquia)

Regra global: sempre adquirir `LOCK_A` antes de `LOCK_B`, em TODAS as threads.

Thread 1 e Thread 2:

```
adquirir(LOCK_A)
adquirir(LOCK_B)
// secao critica
liberar(LOCK_B)
```

`liberar(LOCK_A)`

Regras de entrega

Implementação

A implementação deve ser feita em **Java ou Python**. Não é permitido misturar linguagens entre as partes — escolha uma e use a mesma nas três partes.

Não é permitido utilizar bibliotecas que já resolvam o problema (por exemplo, soluções prontas do Jantar dos Filósofos). É permitido usar a API padrão de threads e semáforos da linguagem escolhida.

O código deve compilar/executar em ambiente padrão (JDK 17+ ou Python 3.10+).

Instruções de execução devem estar no README.

Repositório no GitHub

O grupo deve manter um único repositório **público** no GitHub contendo todo o material do trabalho.

Estrutura mínima sugerida: pastas `parte1-filosophos/`, `parte2-semaforo/`, `parte3-deadlock/`, cada uma com seu(s) arquivo(s) fonte; arquivo `README.md` na raiz.

O README deve conter: nome do grupo e dos integrantes, linguagem escolhida, instruções de compilação e execução, relatório técnico de cada parte, prints/logs de execução e o link do vídeo no YouTube.

Histórico de commits deve refletir a participação de todos os integrantes (cada aluno usando sua própria conta do GitHub). Repositórios com commits concentrados em um único autor poderão ser penalizados.

Vídeo no YouTube

Cada grupo deve gravar e publicar **um único vídeo no YouTube, com no máximo 10 minutos**, em modo **público ou não listado público** (nunca privado), apresentando o trabalho completo.

Todos os integrantes do grupo devem aparecer e falar no vídeo, explicando partes significativas da solução. A simples leitura do código não é suficiente — espera-se que cada integrante demonstre compreensão do que está apresentando.

Recomenda-se compartilhar a tela durante demonstrações de execução, mostrando o código rodando e os logs/saídas relevantes (deadlock travando, semáforo corrigindo a contagem, etc.).

Atente-se à qualidade de áudio (sem ruído excessivo, volume audível) e de imagem (resolução mínima de 720p). Vídeos com qualidade insuficiente sofrerão penalização.

É proibido o uso de vozes sintéticas/clonadas, narração gerada por IA, avatares gerados por IA ou legendas/edição automatizadas por IA. O vídeo deve ser gravado pelos próprios integrantes do grupo, com suas próprias vozes.

O **link do vídeo no YouTube deve constar no README** do repositório, em local visível (recomendado: logo no topo).

Entrega no AVA

Os integrantes devem estar reunidos em um grupo no AVA. Apenas um integrante precisa submeter, mas todos devem estar vinculados ao grupo.

A entrega no AVA deve conter: link do repositório GitHub (público) e link do vídeo no YouTube. Outros materiais ficam no próprio repositório.

Política de uso de IA generativa

É proibido utilizar IA generativa para: (i) escrever, completar ou refatorar o código; (ii) redigir o relatório, o README ou qualquer texto entregue; (iii) elaborar o roteiro ou narração do vídeo; (iv) gerar voz, imagem ou avatar para o vídeo; (v) gerar legendas, traduções ou cortes automáticos do vídeo. **O descumprimento implica zeramento integral da atividade para todos os integrantes do grupo.**

É permitido consultar a documentação oficial das linguagens, materiais didáticos da disciplina, livros, tutoriais escritos por humanos e os colegas do próprio grupo.

Buscadores tradicionais (Google, DuckDuckGo) podem ser usados, mas atalhos que retornam respostas geradas por IA (AI Overviews, Copilot, etc.) não podem ser copiados nem servir de base para o trabalho.

O professor poderá realizar arguições orais individuais a qualquer momento após a entrega. Não conseguir explicar o próprio código ou trechos do próprio relatório é considerado evidência de descumprimento desta política.

Critérios de avaliação

- Clareza conceitual ao explicar impasse e inanição no Jantar dos Filósofos, condição de corrida e o papel dos semáforos, e as quatro condições de Coffman para deadlock.
- Correção do protocolo proposto na Parte 1 (ausência demonstrada de impasse) e uso apropriado de justiça quando relevante para evitar inanição.
- Demonstração reprodutível da condição de corrida na Parte 2 e sua eliminação com semáforo, incluindo discussão de visibilidade/ordenação.
- Reprodução observável do deadlock na Parte 3, mapeamento das quatro condições e correção fundamentada por hierarquia de recursos ou outra técnica equivalente.
- Qualidade do repositório no GitHub: organização, README completo, histórico de commits distribuído entre os integrantes.
- Qualidade do vídeo: áudio claro, imagem com resolução adequada, participação efetiva de todos os integrantes, demonstrações de execução.
- Cumprimento integral da política de IA. Evidência de uso de IA generativa = **nota 0 (zero)** para todos os integrantes.

Materiais conceituais para consulta:

Jantar dos Filósofos — https://en.wikipedia.org/wiki/Dining_philosophers_problem
Deadlock (computer science) — [https://en.wikipedia.org/wiki/Deadlock_\(computer_science\)](https://en.wikipedia.org/wiki/Deadlock_(computer_science))
java.util.concurrent.Semaphore — <https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/Semaphore.html>
threading — Thread-based parallelism (Python) — <https://docs.python.org/3/library/threading.html>
Introduction of Deadlock in OS (GeeksforGeeks) — <https://www.geeksforgeeks.org/operating-systems/introduction-of-deadlock-in-operating-system/>
Dining Philosophers in OS (Scaler) — <https://www.scaler.com/topics/operating-system/dining-philosophers-problem-in-os/>